

21. 速度積分距離ノード

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/distance_node.py
種別	Python
区分	runtime node
役割	vehicle speed を積分して /vehicle/s_km を生成する最小ノード。
起動文脈	measure/live 系で direct distance が無い場合の距離供給。

このファイルを読む理由

vehicle speed を積分して /vehicle/s_km を生成する最小ノード。

- reset_odometry service も持つ。
- 入力は /vehicle/speed_kmh だけ。

呼び出し関係

どこから呼ばれるか

- launch/solar_measurement.launch.py
- mpc_solarcar/live_launch.py

次に読むと理解が進むファイル

- 特記事項なし。

同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
from std_srvs.srv import Trigger

class DistanceNode(Node):
    def __init__(self):
        super().__init__('distance_node')
        self.declare_parameter('max_dt_sec', 2.5)
        self.declare_parameter('publish_rate_hz', 1.0)

        self.max_dt_sec = float(self.get_parameter('max_dt_sec').value)
        publish_rate_hz = float(self.get_parameter('publish_rate_hz').value)

        self.s_km = 0.0
        self.last_time = None
        self.last_speed = math.nan

        self.sub_speed = self.create_subscription(Float32, '/vehicle/speed_kmh', self._on_speed, 10)
        self.pub_s = self.create_publisher(Float32, '/vehicle/s_km', 10)
        self.srv_reset = self.create_service(Trigger, '/vehicle/reset_odometry', self._on_reset)

        self.timer = self.create_timer(1.0 / publish_rate_hz, self._publish)
        self.get_logger().info('DistanceNode started.')
```

主要構造の読み取り

主要クラスは DistanceNode。主要関数は main。ROS パラメータ宣言は 2 件。ROS I/O は publisher 1 件、subscription 1 件。

主要クラス・関数

行	種別	名前	補足
9	class	DistanceNode	bases: Node
10	function	__init__	args: self
29	function	_on_speed	args: self, msg
40	function	_on_reset	args: self, request, response
48	function	_publish	args: self
54	function	main	

ファイルを上から読んだときの定義順

行	内容
9	クラス DistanceNode を定義する。
54	関数 main を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>import math</code>	<code>clip</code> , <code>sqrt</code> , <code>sin/cos</code> 、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L20, L32, L43。
3	<code>import rclpy</code>	ROS 2 Python ノードとして起動・ <code>spin</code> するため。このファイル内での主な使用位置は L55, L57, L59。
4	<code>from rclpy.node import Node</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L9。
5	<code>from std_msgs. msg import Float32</code>	<code>Float32</code> や <code>String</code> など軽量メッセージで <code>planner / vehicle</code> 値を <code>publish</code> するため。このファイル内での主な使用位置は L22, L23, L29, L49。
6	<code>from std_srvs. srv import Trigger</code>	<code>std_srvs.srv</code> から <code>Trigger</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L24。

関数・クラスを上から順に解説

L9 クラス `DistanceNode`

- 定義: `DistanceNode(bases=Node)`
 - `docstring`: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `__init__`, `create_publisher`, `create_service`, `create_subscription`, `create_timer`, `declare_parameter`, `float`, `get_clock`, `get_logger`, `get_parameter`, `info`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. 関数 `__init__` を定義する。
 2. 関数 `_on_speed` を定義する。
 3. 関数 `_on_reset` を定義する。
 4. 関数 `_publish` を定義する。

L54 関数 `main`

- 定義: `main()`
 - `docstring`: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `DistanceNode`, `destroy_node`, `init`, `shutdown`, `spin`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. `rclpy.init(...)` を実行する。
 2. `node` に `DistanceNode()` の結果を代入する。
 3. `rclpy.spin(...)` を実行する。
 4. `node.destroy_node(...)` を実行する。
 5. `rclpy.shutdown(...)` を実行する。

設定値と入力パラメータ

行	名前	既定値・補足
12	max_dt_sec	2.5
13	publish_rate_hz	1.0

ROS topic I/O

行	種別	topic	callback / 補足
23	publisher	/vehicle/s_km	publisher
22	subscription	/vehicle/speed_kmh	self._on_speed

処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/distance_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。